

TP 02 - Todo List

Ajoutez uniquement les fonctionnalités demandées à l'application existante de fin de jour 1 en React + TypeScript.

Livrable attendu

Une Todo List fonctionnelle intégrée à l'application existante.

Contraintes

Ne pas repartir de zéro. Ne pas casser l'existant. Rester en **.tsx**.

Travail demandé

- Ajouter une tâche avec un **formulaire contrôlé**.
- Modifier une tâche **directement dans la liste**.
- Marquer une tâche comme terminée / non terminée.
- Supprimer une tâche.
- Afficher un **compteur** de tâches restantes.
- Filtrer l'affichage : **toutes**, **actives**, **terminées**.

Point de départ à intégrer dans l'application

Ajoutez ce state ou son équivalent dans votre application existante.

```
type Todo = {
  id: number;
  title: string;
  done: boolean;
};

const [todos, setTodos] = useState([
  { id: 1, title: "Apprendre React", done: false },
  { id: 2, title: "Créer un composant", done: true },
  { id: 3, title: "Maîtriser les hooks", done: false },
]);

const [filter, setFilter] = useState<"all" | "active" | "done">("all");
const [newTitle, setNewTitle] = useState("");
```

Fonctionnalités à implémenter

Fonctionnalité	Attendu fonctionnel
Ajout	Un champ texte et un bouton permettent d'ajouter une tâche. Une saisie vide est refusée. Le cha
Toggle	Chaque tâche peut être cochée / décochée pour changer son état done.
Suppression	Chaque tâche possède une action de suppression qui la retire immédiatement de la liste.
Édition inline	Une tâche peut être modifiée directement dans la liste sans recharger la page. Une valeur vide n
Compteur	Le nombre de tâches non terminées est affiché clairement.

Fonctionnalité	Attendu fonctionnel
Filtre	L'utilisateur peut afficher toutes les tâches, seulement les actives ou seulement les terminées.

Données dérivées attendues

```
const filteredTodos = todos.filter((todo) => {
  if (filter === "active") return !todo.done;
  if (filter === "done") return todo.done;
  return true;
});

const remaining = todos.filter((todo) => !todo.done).length;
```

Contraintes de réalisation

- Ne supprimez pas inutilement la structure existante du projet.
- N'écrivez pas une nouvelle application à côté : faites évoluer l'existante.
- Ne modifiez jamais directement le tableau **todos** ni ses objets.
- Utilisez des mises à jour immuables avec **map**, **filter** et
- Le filtre change seulement l'affichage, pas les données sources.

Actions CRUD attendues

Action	Ce qu'elle doit faire
addTodo	Créer une nouvelle tâche avec un id unique, un titre nettoyé avec trim(), et done à false.
toggleTodo	Inverser la valeur done de la tâche ciblée.
deleteTodo	Retirer la tâche correspondant à l'id.
editTodo	Remplacer le titre d'une tâche par un nouveau titre valide.

Exemples de logique attendue

```
const addTodo = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newTitle.trim()) return;

  setTodos((prev) => [
    ...prev,
    { id: Date.now(), title: newTitle.trim(), done: false }
  ]);

  setNewTitle("");
};

const toggleTodo = (id: number) =>
  setTodos((prev) =>
    prev.map((t) => (t.id === id ? { ...t, done: !t.done } : t))
  );

const deleteTodo = (id: number) =>
  setTodos((prev) => prev.filter((t) => t.id !== id));

const editTodo = (id: number, title: string) =>
  setTodos((prev) =>
    prev.map((t) => (t.id === id ? { ...t, title } : t))
  );
```

Validation

- L'application existante fonctionne toujours.
- Toutes les actions demandées sont opérationnelles.
- Le code compile sans erreur TypeScript.
- Le compteur et le filtre donnent un résultat juste.
- Aucune mutation directe du state n'est utilisée.

Bonus possibles

Ajouter un message quand la liste est vide, un bouton "supprimer les tâches terminées", ou une persistance simple dans le localStorage.